

Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação

SSC0903 – Computação de Alto Desempenho

PRIMEIRO TRABALHO PRÁTICO (TB1)

Simulação Paralela da Propagação Direcional de Incêndio com Zonas de Contenção

NOME	NUSP
Gabriel de Araujo Lima	14571376
Luis Henrique Ponciano	15577760
Murilo Leandro Garcia	15480943
Raul Ribeiro Teles	13688176
Renan Banci Catarin	14658181

São Carlos – SP
2026

Sumário

1	Introdução	3
2	Descrição da Solução Sequencial	4
2.1	Estruturas de dados e representação da matriz	4
2.2	Leitura e validação da entrada	4
2.3	Geração do terreno e inicialização dos focos e da contenção	4
2.4	Estrutura do laço principal	4
2.5	Cálculo do potencial de ignição e regras de transição de estado	5
2.6	Estatísticas, checksum e medição de tempo	5
3	Descrição da Solução Paralela	5
3.1	Visão geral da estratégia de paralelização	5
3.2	Layout de dados: Struct of Arrays (SoA)	6
3.3	Paralelização da ativação de zonas	7
3.4	Partes mantidas sequenciais: verificação de fogo inicial e geração do terreno	7
3.5	Paralelização da atualização da matriz	7
3.6	Uso de SIMD	8
3.7	Reduções para os contadores	8
3.8	Ausência de condições de corrida e troca segura das matrizes	9
3.9	Compartilhamento da condição de parada	9
3.10	<code>default(none)</code> e cláusulas de escopo de dados	9
3.11	Ausência de <code>critical/atomic</code> no laço principal	9
3.12	Determinismo independente do número de threads	10
3.13	Delimitação do trecho cronometrado	10
4	Ambiente Experimental	11
5	Validação das versões	11
6	Resultado de Desempenho	11
6.1	Tempos de Execução	11
6.2	Speedup e Eficiência	11
6.3	Gráficos	13
6.3.1	Carga grande	13
6.3.2	Carga Média	14
6.3.3	Carga Pequena	15
7	Análise dos Resultados	16
7.1	Entrada de Carga Pequena	16
7.1.1	Tempo de execução	16
7.1.2	Speedup	16
7.1.3	Eficiência	17
7.1.4	Comparação entre os schedules	17
7.2	Entrada de Carga Média	17
7.2.1	Tempo de execução	17
7.2.2	Speedup	18
7.2.3	Eficiência	18

7.3	Entrada de Carga Grande	18
7.3.1	Tempo de execução	18
7.3.2	Speedup	18
7.3.3	Eficiência	19
7.4	Escalabilidade	19

1 Introdução

Este trabalho tem como objetivo implementar e avaliar duas versões de um simulador de propagação direcional de incêndio com zonas de contenção: uma versão sequencial, escrita em linguagem C, e uma versão paralela, escrita em C com a API OpenMP. Ambas devem produzir exatamente o mesmo resultado para uma mesma entrada, diferindo apenas no tempo de execução. A simulação é determinística e baseada em um autômato celular sobre uma matriz retangular de células, cada uma com tipo de cobertura, umidade, estado e tempo de queima. A propagação do fogo considera a direção e a intensidade do vento, bem como a posição relativa dos vizinhos em chamas, seguindo as regras definidas no enunciado do projeto.

Dada uma matriz de $L \times C$ células, uma configuração de vento, um conjunto de focos iniciais de incêndio e um conjunto de zonas de contenção ativadas em passos pré-determinados, deseja-se simular a evolução do incêndio ao longo de até P passos de tempo e computar um conjunto de estatísticas ao final da execução, incluindo o número de passos executados, a quantidade final de células em cada estado, o total de ignições, o pico de ignições, os percentuais de área queimada e protegida, e um *checksum* da configuração final. A versão paralela deve explorar o paralelismo de dados inerente à atualização das células, empregando uma região paralela persistente.

2 Descrição da Solução Sequencial

A versão sequencial, implementada em `fire_seq.c`, segue estritamente a ordem de operações definida no enunciado, sem qualquer diretiva OpenMP de paralelização (a biblioteca é usada apenas para medição de tempo, via `omp_get_wtime()`).

2.1 Estruturas de dados e representação da matriz

A matriz $L \times C$ é armazenada de forma linear, com o índice de cada célula calculado pela macro `IDX(l, c, max_c) = l * max_c + c`, conforme exigido na Seção 6.1 do enunciado. A struct `Terreno` concentra os vetores de cobertura, umidade, mapa de ativação de contenção e, principalmente, dois pares de vetores para viabilizar a técnica de *double buffering*: `estado/proximo_estado` e `tempo_atual/proximo_tempo`. Isso garante que o cálculo do próximo passo dependa exclusivamente dos valores do passo atual, como pedido na Seção 7.

2.2 Leitura e validação da entrada

A função `ler_entrada` lê, em ordem, a configuração geral (L , C , passos máximos, threads, seed, limiar), a configuração do vento e a quantidade/descrição de focos e zonas de contenção. Em seguida, `validar_entrada` verifica todas as restrições da Seção 5: valores positivos de L , C , threads e limiar, $P \geq 0$, componentes do vento em $[-1, 1]$ e não simultaneamente nulas, intensidade em $[0, 5]$, zonas dentro dos limites da matriz e com `passo_ativacao` válido, e ausência de focos repetidos ou fora da matriz. A validação de focos sobre células não combustíveis é feita à parte, em `validar_focos_iniciais`, pois depende da cobertura já gerada.

2.3 Geração do terreno e inicialização dos focos e da contenção

A função `gerar_terreno` percorre a matriz em ordem crescente de índice linear, gerando para cada célula, nesta ordem, a cobertura (`rand_r(&seed) % 100`) e, imediatamente após, a umidade (`rand_r(&seed) % 101`), exatamente como descrito na Seção 6.3. Essa ordem estrita é o que garante que as versões sequencial e paralela produzam o mesmo terreno para a mesma seed, já que a geração continua sendo feita por uma única thread também na versão paralela.

`iniciar_fogos` marca as células dos focos iniciais como `EM_CHAMAS` e atribui o tempo de queima inicial (`tempo_inicial`: 2 para vegetação rasteira, 4 para floresta). `construir_contencao` inicializa o vetor `ativacao` com -1 e, para cada zona, preenche o retângulo definido por $(linha_ini, coluna_ini) - (linha_fim, coluna_fim)$ com o passo de ativação, mantendo o menor passo em caso de sobreposição (Seção 6.6). Essa etapa ocorre fora do trecho cronometrado, conforme exigido.

2.4 Estrutura do laço principal

A função `simular` implementa exatamente a sequência de 5 etapas da Seção 7.1 dentro do laço de passos: (1) ativação de zonas e (2) cálculo do próximo estado são feitos juntos, célula a célula, por `processar_celula`; (3) as estatísticas do passo (ignições) são acumuladas durante essa varredura; (4) a troca das matrizes de estado e de tempo é feita por troca de ponteiros; e (5) a condição de parada é verificada pela flag `pegando_fogo`, que indica se ainda há alguma célula em chamas ou entrando em chamas. Antes do laço, `verificar_fogo` trata

o caso de nenhuma célula em chamas após a inicialização, retornando `passos = 0` sem executar nenhum passo (Seção 9).

2.5 Cálculo do potencial de ignição e regras de transição de estado

A função `potencial_ignicao` implementa fielmente a Seção 8: percorre os 8 vizinhos de Moore, ignorando os que estão fora da matriz; para cada vizinho em chamas, calcula o sentido de propagação (`prop_l`, `prop_c`), classifica o peso básico (10 ortogonal, 7 diagonal) e o alinhamento com o vento ($A \in \{-2, \dots, 2\}$); o peso do vizinho é $\max(1, P_{\text{básico}} + \text{intensidade} \times A)$, somado a S . O potencial final é $I = \lfloor S \times \text{fator_combustivel} \times (100 - \text{umidade})/100 \rfloor$, usando exclusivamente aritmética inteira, como exigido.

Em `processar_celula`: células ativadas por uma zona no passo corrente e que estejam `INTACTA` tornam-se `CONTENCAO` (nunca sobrescrevendo uma célula já em chamas, ou seja, respeitando a regra de que contenção não apaga incêndio); células `INTACTA` com potencial $\geq \text{LIMIAR}$ tornam-se `EM_CHAMAS`, recebendo o tempo inicial de queima; células `EM_CHAMAS` têm o tempo decrementado e passam a `QUEIMADA` quando o tempo chega a zero; células `QUEIMADA`, `CONTENCAO` e `NAO_COMBUSTIVEL` permanecem inalteradas, pois `proximo_estado/proximo_tempo` são inicializados como cópia do estado atual no início da função.

2.6 Estatísticas, checksum e medição de tempo

A cada passo, o número de novas ignições é somado em `s->ignicoes` e comparado ao maior valor já visto para atualizar `max_ignicoes` e `passo_mais_ignicoes` (Seção 10). Ao final, `imprimir_resultados` calcula os percentuais de área queimada e protegida em relação ao total de células combustíveis iniciais (`s->combustiveis`, calculado por `contar_combustiveis` antes da simulação) e o checksum, seguindo exatamente a fórmula da Seção 10 ($\text{estado} * 31 + \text{tempo}$, em ordem linear). Tanto o checksum quanto o cálculo dos percentuais e a impressão ficam fora do trecho cronometrado: a medição com `omp_get_wtime()` em `simular` envolve apenas a ativação de zonas, o cálculo de propagação, a atualização de estados/tempos, a contagem de estatísticas do passo, a troca de matrizes e a verificação da condição de parada, conforme a Seção 12.

Além disso, por não haver qualquer paralelização, a ordem de leitura e escrita das células é estritamente sequencial e determinística. Essa versão serve, portanto, como *referência* para validar a corretude da versão paralela (Seção 5 do relatório), já que ambas devem produzir os mesmos valores de saída para a mesma entrada, exceto o tempo de execução.

3 Descrição da Solução Paralela

A versão paralela foi implementada em C/OpenMP a partir da mesma base lógica da versão sequencial, alterando apenas a função `simular`. Todas as demais etapas (leitura, validação, geração do terreno, focos e mapa de contenção) permanecem sequenciais, pois estão fora do trecho cronometrado e a geração do terreno precisa manter a ordem estrita de chamadas a `rand_r` para garantir reprodutibilidade.

3.1 Visão geral da estratégia de paralelização

A unidade de paralelismo é a célula: cada iteração do laço sobre as $L \times C$ células é independente dentro de um mesmo passo, pois `processar_celula` só lê o estado do passo atual

(estado, tempo_atual) e escreve exclusivamente na posição i dos vetores de próximo estado. Para evitar o custo de criar e destruir threads a cada passo, a região paralela é aberta *uma única vez*, envolvendo todo o laço de passos:

```
#pragma omp parallel num_threads(c->threads) default(none) \
    shared(continuar, passo, n_cells, t, c, s, pegando_fogo, ignicoes_passo)
while (passo < c->max_passos && continuar) {
    #pragma omp for simd schedule(static) \
        reduction(+:ignicoes_passo) reduction(max:pegando_fogo)
    for (long long i = 0; i < n_cells; i++) {
        processar_celula(t, c, i, passo, &ignicoes_passo, &pegando_fogo);
    }
    #pragma omp single
    { /* estatísticas, troca de matrizes, atualização de passo/continuar */ }
}
```

As T threads são criadas uma vez e reutilizadas em todos os passos da simulação, com o laço `while` executando *dentro* da região paralela.

3.2 Layout de dados: Struct of Arrays (SoA)

A malha é representada como *struct of arrays* (SoA), e não como *array of structs* (AoS): a struct `Terreno` guarda um vetor separado por atributo (`cobertura`, `umidade`, `ativacao`, `estado`, `proximo_estado`, `tempo_atual`, `proximo_tempo`), todos indexados pela mesma posição linear `IDX(1, c, C)`, em vez de um único vetor de uma struct `Celula` contendo todos os atributos de uma célula agrupados:

```
/* SoA (Structure of Arrays) | adotado */
typedef struct {
    unsigned char *cobertura;
    unsigned char *umidade;
    int           *ativacao;
    unsigned char *estado;
    unsigned char *proximo_estado;
    unsigned char *tempo_atual;
    unsigned char *proximo_tempo;
} Terreno;
```

Essa escolha traz vantagens ligadas à estratégia de paralelização:

- **Localidade espacial e uso eficiente de cache:** como o `omp for` divide o laço de células em blocos contíguos de índices (`schedule(static)` ou `schedule(dynamic, 512)`), cada thread acessa uma faixa contígua de cada vetor. Em um layout AoS, os campos de uma mesma célula ficariam agrupados, mas células vizinhas do mesmo bloco ficariam espalhadas por múltiplas linhas de cache maiores (cada `Celula` ocupando mais bytes); no SoA, threads processando células consecutivas leem posições consecutivas de *cada* vetor, maximizando o aproveitamento de cada linha de cache carregada.
- **Carregamento apenas dos dados necessários:** funções como `potencial_ignicao` usam somente `estado`, `cobertura` e `umidade`, sem tocar em `ativacao` ou nos vetores

de tempo. Em SoA, apenas os vetores efetivamente lidos ocupam espaço em cache durante essa fase; em AoS, cada acesso a uma célula traria, na mesma linha de cache, campos que aquela função não usa (como `ativacao`), desperdiçando memória.

- **Vetorização eficiente (SIMD):** A diretiva `omp for simd` permite combinar a distribuição das iterações entre threads com a exploração de paralelismo SIMD. O layout SoA favorece essa estratégia ao fornecer acessos contíguos aos atributos armazenados em cada vetor.

Esse layout é mantido também para os vetores de "próximo" estado e tempo, preservando o mesmo padrão de acesso contíguo tanto na leitura do passo atual quanto na escrita do próximo passo.

3.3 Paralelização da ativação de zonas

A ativação das zonas de contenção não recebeu um laço paralelo separado: ela é feita dentro da própria `processar_celula`, testando `t->ativacao[i] == passo` para cada célula i . Como essa checagem já ocorre dentro do `omp for` que percorre todas as células a cada passo, a ativação das zonas é paralelizada naturalmente, célula a célula, junto com o cálculo do próximo estado — sem laço ou sincronização adicional.

3.4 Partes mantidas sequenciais: verificação de fogo inicial e geração do terreno

A verificação de existência de fogo após a inicialização (`verificar_fogo`) é feita *sequencialmente*, por uma única thread, imediatamente após o início da medição de tempo e antes da abertura da região `#pragma omp parallel`. Se nenhuma célula estiver em chamas, a função retorna imediatamente com `s->passos = 0`, sem que nenhum passo seja executado e sem que a região paralela chegue a processar qualquer célula, em conformidade com a Seção 9 do enunciado. Por ser um único laço de varredura, curto perto do custo da simulação completa, não há benefício relevante em paralelizá-lo, e mantê-lo sequencial evita overhead de criação de threads para o caso trivial de simulação sem fogo.

Por sua vez, a geração de cobertura e umidade (`gerar_terreno`) permanece sequencial em ambas as versões, por exigência explícita da Seção 6.3 do enunciado: a cobertura e a umidade de cada célula devem ser geradas nesta ordem exata, célula a célula, pois `rand_r` produz uma sequência pseudoaleatória *dependente do estado anterior* do gerador. Paralelizar esse laço quebraria a ordem determinística de consumo da semente, fazendo com que o terreno gerado (e, consequentemente, todo o resultado da simulação) dependesse do número de threads e da ordem de escalonamento — o que violaria o requisito de que as versões sequencial e paralela produzam exatamente o mesmo resultado para a mesma entrada (Seção 1).

3.5 Paralelização da atualização da matriz

O laço sobre as n_cells células é paralelizado com `#pragma omp for`, unindo a divisão de trabalho entre as T threads da região persistente já aberta. Foram testados dois schedules:

- `schedule(static)` em `fire_omp.c`: divide o intervalo de índices em blocos contíguos e aproximadamente iguais entre as threads, com baixo overhead de escalonamento,

adequado quando o custo de `processar_celula` é relativamente uniforme entre as células.

- `schedule(dynamic, 512)` em `fire_omp_dynamic.c`: distribui blocos de 512 células sob demanda, o que tende a balancear melhor a carga quando o custo de `potencial_ignicao` varia (células isoladas custam menos que células cercadas de vizinhos em chamas), ao custo de maior overhead de sincronização para obter novos blocos.

A comparação de desempenho entre os dois schedules é apresentada futuramente.

3.6 Uso de SIMD

A diretiva `omp for simd` foi aplicada ao laço sobre as células (o laço que percorre os `n_cells` índices i dentro de cada passo), e não aos laços internos de `potencial_ignicao` sobre os 8 vizinhos de Moore, combinando a divisão de trabalho entre threads (`for`) com a vetorização das iterações dentro de cada thread (`simd`), atendendo à exigência do enunciado de uso de `simd` na versão paralela.

Essa vetorização, porém, esbarra na divergência condicional de `processar_celula`: o resultado de cada `if` (ativação de zona, estado atual, vizinhos em chamas) varia célula a célula. Como instruções SIMD aplicam a mesma operação a várias células simultaneamente, um `if` divergente obriga o processador a calcular ambas as alternativas para o grupo inteiro e descartar, ao final, o resultado que não se aplicava a cada célula – o que reduz o ganho esperado nesses trechos.

Por outro lado, os vetores envolvidos (`estado`, `tempo_atual`, `cobertura`) usam `unsigned char`, um tipo pequeno e armazenado de forma contígua e sequencial no layout SoA. Isso permite que mais elementos caibam em uma mesma instrução vetorial, o que pode compensar boa parte do custo extra gerado pela divergência dos `ifs` e aumentar o throughput geral do laço.

Ainda assim, a diretiva não compromete a corretude – o pior caso equivale a não vetorizar esses trechos – e permite ao compilador aproveitar automaticamente os subtrechos mais favoráveis, sem exigir reestruturação do código.

3.7 Reduções para os contadores

Dois contadores são combinados entre threads exclusivamente por meio de `reduction`, nunca por `critical` ou `atomic`:

- `reduction(+:ignicoes_passo)`: soma simples do total de novas ignições no passo, associativa e comutativa, independente da ordem de execução das threads.
- `reduction(max:pegando_fogo)`: como `pegando_fogo` é uma flag booleana (0 ou 1) sinalizando se *alguma* célula continua queimando ou entrou em chamas, o operador `max` funciona como um substituto correto para um “ou lógico” entre threads — se qualquer thread definir `pegando_fogo = 1` para alguma célula, o valor combinado final será 1.

Não foi necessária uma redução customizada para o pico de ignições: como a paralelização atua sobre as *células* de um mesmo passo, e não sobre os passos em si, a parte custosa — que escala com o número de células — já está coberta pela redução de `ignicoes_passo`. A atualização de `passo_mais_ignicoes` é apenas uma comparação simples, feita uma única vez por passo, a partir do valor já reduzido; não há trabalho proporcional a $L \times C$ associado

a ela. Executá-la dentro da seção `single`, por uma única thread, evita que múltiplas threads repitam a mesma comparação sobre o mesmo dado sem nenhum ganho de desempenho.

3.8 Ausência de condições de corrida e troca segura das matrizes

Dentro do `omp for`, cada thread escreve exclusivamente nos índices i que lhe foram atribuídos pelo schedule, em posições disjuntas de `proximo_estado` e `proximo_tempo`; não há sobreposição de escrita entre threads. Os vetores `estado`, `tempo_atual`, `cobertura`, `umidade` e `ativacao` são apenas lidos durante o processamento do passo (inclusive nas consultas aos vizinhos de Moore em `potencial_ignicao`, que podem ler células atribuídas a outras threads, mas somente em modo leitura), o que elimina qualquer condição de corrida read/write ou write/write.

A troca de ponteiros entre `estado`↔`proximo_estado` e `tempo_atual`↔`proximo_tempo` é realizada dentro de `#pragma omp single`, executada por apenas uma thread. A segurança vem das barreiras implícitas: o `omp for` sem cláusula `nowait` garante que todas as threads concluam a escrita do passo antes de qualquer uma entrar na seção `single`, e a barreira implícita ao final do `single` garante que a troca (e a atualização de `passo/continuar`) seja visível a todas as threads antes de reavaliarem a condição do `while` e iniciarem o próximo passo.

3.9 Compartilhamento da condição de parada

A flag `continuar` é declarada `shared` e só é modificada dentro da seção `single`, a partir do valor já reduzido de `pegando fogo` (`if (!pegando_fogo) continuar = 0;`). Como essa atribuição ocorre antes da barreira implícita de saída do `single`, todas as threads enxergam o valor atualizado ao reavaliarem `while(passo < c->max_passos && continuar)` no início da próxima iteração, sem necessidade de `critical` ou `atomic` para propagar a condição de parada.

3.10 `default(none)` e cláusulas de escopo de dados

A região paralela usa `default(none)`, obrigando a listar explicitamente todas as variáveis: `shared(continuar, passo, n_cells, t, c, s, pegando_fogo, ignicoes_passo)`. O índice de laço `i` é automaticamente privado ao `omp for`, e `ignicoes_passo/pegando_fogo`, apesar de compartilhadas na região externa, se tornam variáveis de redução (com cópias privadas por thread) somente dentro do escopo do `omp for`, conforme exigido pela semântica de `reduction` do OpenMP.

3.11 Ausência de `critical/atomic` no laço principal

Nenhuma diretiva `critical` ou `atomic` é usada no núcleo do laço (o `omp for simd` sobre as células). Toda a necessidade de sincronização e agregação de resultados foi resolvida com `reduction` (contadores) e `single` (uma seção curta, executada uma vez por passo, fora do laço quente de $L \times C$ células), evitando o custo de sincronização por célula que `critical/atomic` implicariam.

3.12 Determinismo independente do número de threads

O resultado final independe de T e do schedule escolhido pelos seguintes motivos: (i) cada célula lê exclusivamente o estado do passo anterior, nunca o de outra célula sendo escrita no mesmo passo; (ii) a soma de `ignicoes_passo` e o `max` de `pegando_fogo` são operações associativas e comutativas, produzindo o mesmo resultado independentemente da ordem ou do agrupamento das contribuições de cada thread; (iii) a atualização de `passo_mais_ignicoes` é feita sequencialmente na seção `single`, a partir do valor já combinado. Isso garante que `fire_omp` produza exatamente as mesmas estatísticas e o mesmo checksum que `fire_seq`, para qualquer T e qualquer schedule testado.

3.13 Delimitação do trecho cronometrado

A medição com `omp_get_wtime()` envolve exatamente o mesmo trecho da versão sequencial: inicia após a verificação de fogo inicial (`verificar_fogo`) e termina imediatamente após o laço `while` de passos, cobrindo a ativação de zonas, o cálculo de propagação, a atualização de estados/tempos, a agregação de estatísticas do passo, a troca de matrizes e a verificação da condição de parada — sem incluir leitura, validação, geração do terreno, cálculo do checksum ou impressão dos resultados, que continuam sequenciais em `imprimir_resultados`.

4 Ambiente Experimental

Os testes foram executados em um computador com processador Intel i5-8500 de 6 núcleos físicos e 6 threads (1 thread por núcleo), 16 GB de memória RAM e sistema operacional Manjaro Linux (Release 26.1.2) com Kernel 6.12.108-1-MANJARO x86_64. Os programas em C foram compilados usando o gcc na versão 16.2.1.

O ambiente experimental foi configurado para executar as simulações sequencial e paralela utilizando três conjuntos de dados: `entrada_carga_pequena.txt`, `entrada_carga_media.txt` e `entrada_carga_grande.txt`.

O número de *threads* testadas na versão paralelizada foi 1, 2, 4, 8, e 16, editando temporariamente as entradas através de um script automatizado. Para mitigar oscilações inerentes ao escalonador do sistema operacional, o script automatizado executou cada contexto três vezes independentes, extraindo-se a média aritmética do tempo de processamento.

5 Validação das versões

Para validar se a versão paralela estava certa, comparamos os resultados dela com os resultados da versão sequencial. A simulação é determinística, então os resultados precisam ser exatamente iguais. Checamos se todos os valores da saída eram idênticos: quantidade de passos, quantidade de células não combustíveis, intactas, em chamas, queimadas e de contenção, além do total de ignições, do pico de ignições e do checksum final. Os valores bateram perfeitamente em todos os testes, o que mostra que a versão paralela funciona direito e não tem erros de concorrência.

6 Resultado de Desempenho

6.1 Tempos de Execução

A versão sequencial obteve tempos de execução de 0.781 segundos na carga pequena, 8.845 segundos na carga média e 30.941 segundos na carga grande. A paralelização reduziu o tempo de execução conforme o número de threads aumentou, atingindo o menor tempo absoluto com 8 threads. Na carga grande, o esquema dinâmico reduziu o tempo de 30.94 segundos (sequencial) para 8.03 segundos com 4 threads e para 5.96 segundos com 8 threads.

6.2 Speedup e Eficiência

O *speedup* (S_p) mede o ganho de desempenho obtido ao paralelizar a execução de um algoritmo, sendo calculado pela razão entre o tempo de execução da versão sequencial (T_{seq}) e o tempo de execução da versão paralela (T_{par}) com p threads:

$$S_p = \frac{T_{seq}}{T_{par}} \quad (1)$$

A eficiência (E_p) indica o grau de aproveitamento dos recursos computacionais empregados, sendo calculada pela razão entre o *speedup* obtido (S_p) e o número de threads (p):

$$E_p = \frac{S_p}{p} = \frac{T_{seq}}{p \times T_{par}} \quad (2)$$

O speedup e a eficiência mostraram que a versão dinâmica teve o melhor desempenho. O speedup máximo foi de 5.57x na carga pequena, 5.48x na carga média e 5.19x na carga grande, todos usando 8 threads com o esquema dinâmico.

Nas diferentes entradas, a eficiência ficou perto de 1.0 usando até 4 threads. Com 8 threads, ela cai para aproximadamente 0.70, e com 16 threads chega a 0.31. Isso se reflete nos gráficos de speedup, que mantém relação linear até 4 threads, e depois disso começa a atingir um platô.

6.3 Gráficos

6.3.1 Carga grande

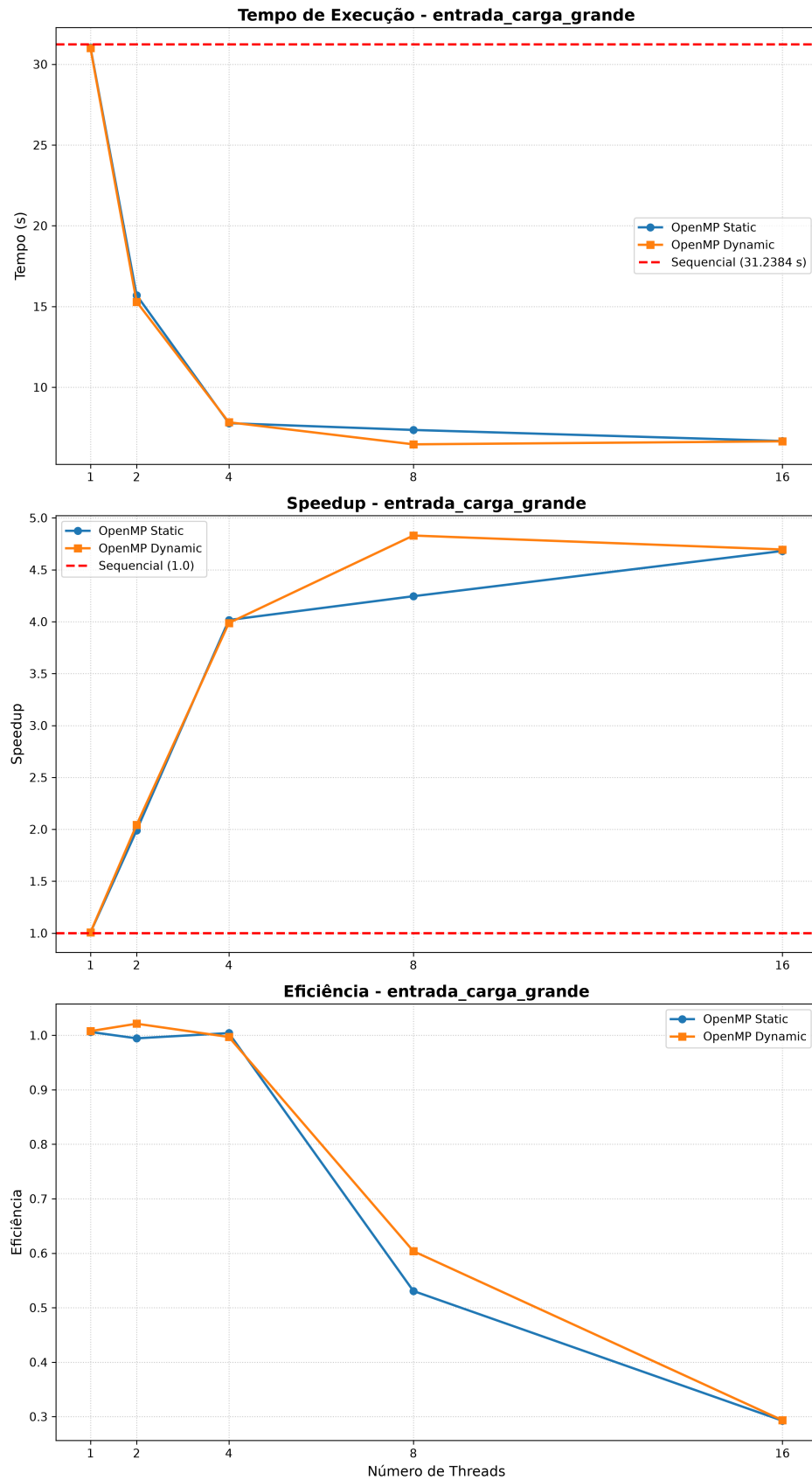


Figure 1: Métricas de desempenho - entrada_carga_grande.

6.3.2 Carga Média

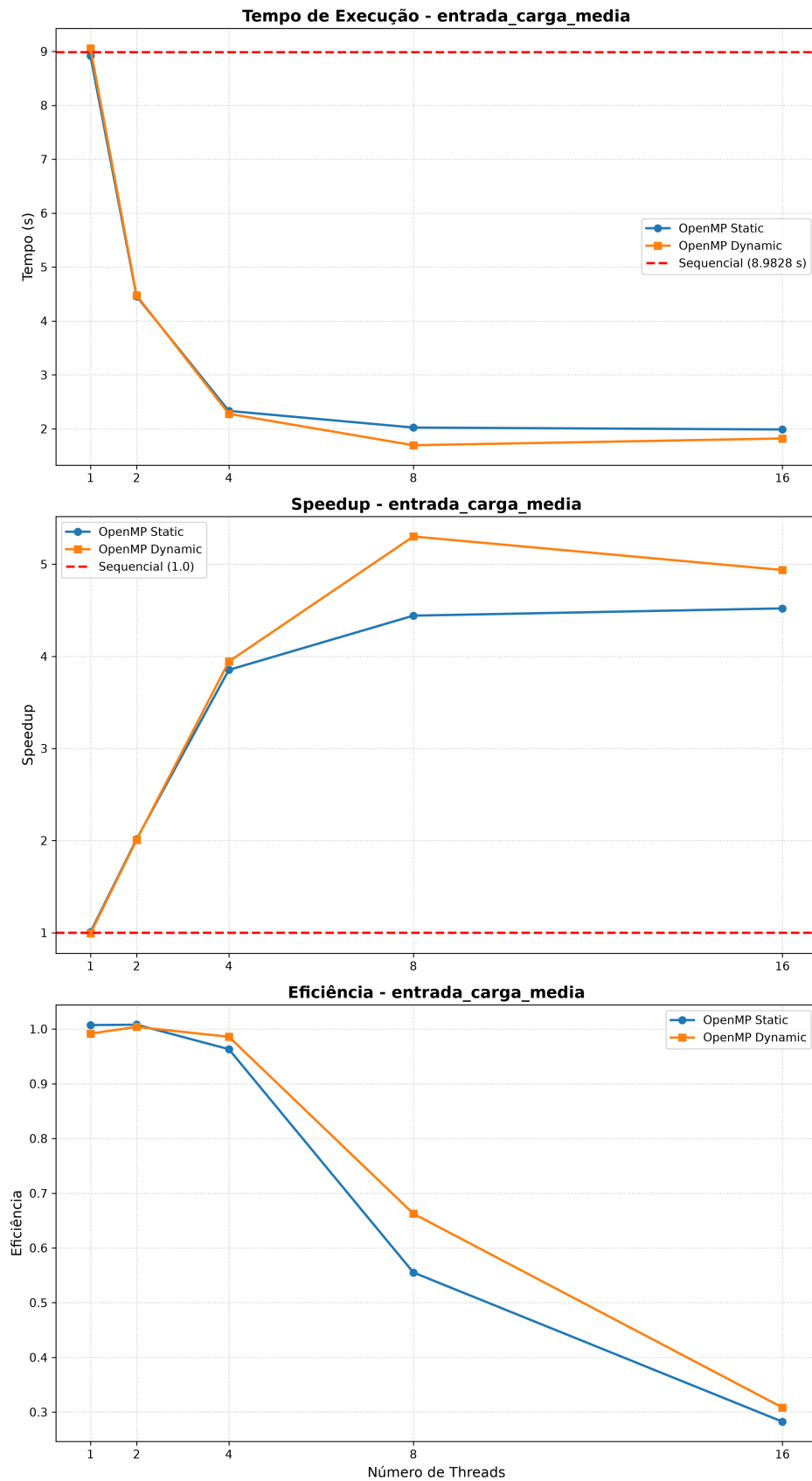


Figure 2: Métricas de desempenho - entrada_carga_media.

6.3.3 Carga Pequena

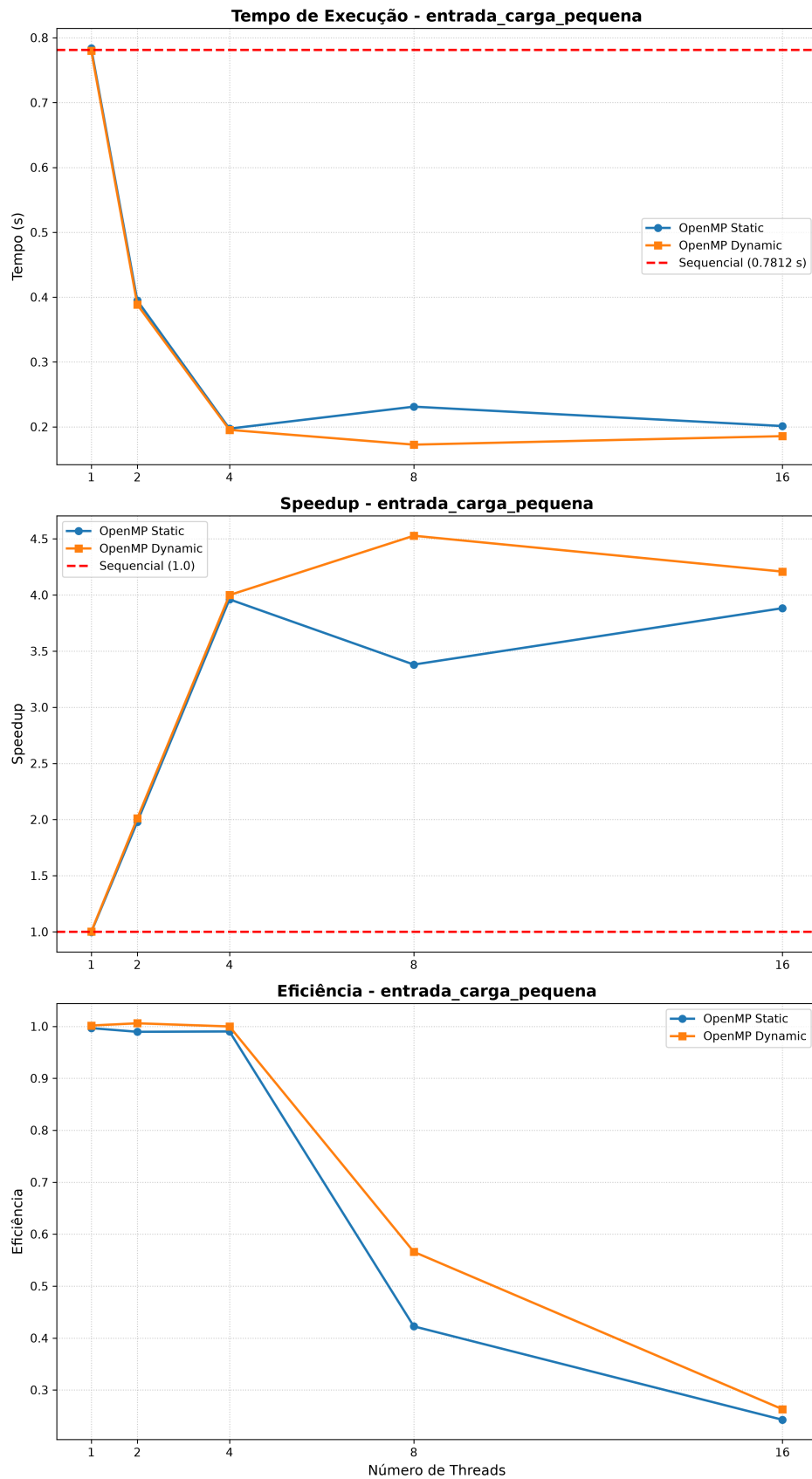


Figure 3: Métricas de desempenho - entrada_carga_pequena.

7 Análise dos Resultados

7.1 Entrada de Carga Pequena

A entrada de carga pequena (400×500 células, $P_{max} = 80$) é a que exige a menor quantidade de trabalho por passo entre as três cargas testadas, o que a torna a mais sensível a efeitos de overhead de paralelização e a variações do escalonador do sistema operacional.

7.1.1 Tempo de execução

A Figura 3 mostra que, com uma única thread, ambos os schedules reproduzem essencialmente o tempo sequencial (0.781 s), como esperado, já que com $T = 1$ não há divisão de trabalho nem sincronização entre threads. O tempo cai de forma acentuada até 4 threads, ponto em que **static** e **dynamic** atingem cerca de 0.20 s – uma redução de quase 4x –, indicando que, até esse número de threads, o ganho de paralelismo ainda supera com folga o overhead introduzido pela região paralela e pela barreira/**single** a cada passo.

A partir de 8 threads, porém, os dois schedules se comportam de forma bem diferente. O **dynamic** continua a melhorar e atinge seu menor tempo absoluto (por volta de 0.140 s) exatamente em 8 threads, enquanto o **static** passa a *piorar* (de aproximadamente 0.20 s para 0.21 s) ao sair de 4 para 8 threads. Em 16 threads, os dois schedules se movem em direções opostas: o **dynamic** sobe para cerca de 0.158 s – uma regressão de aproximadamente 12,8% em relação aos 8 threads –, enquanto o **static** recua para cerca de 0.196 s – uma melhora de cerca de 7% em relação a 8 threads, chegando inclusive a um tempo menor do que o obtido com 4 threads. Ou seja, a partir de 8 threads não há um ganho consistente para os dois schedules: o **dynamic** piora em 16 threads, enquanto o **static** apresenta uma pequena recuperação, de cerca de 7%, em relação a 8 threads. Esse comportamento é coerente com o fato de a máquina possuir apenas 6 núcleos físicos: a partir de 8 threads, o sistema já está em *oversubscription*, e os custos associados ao escalonamento, à sincronização e à disputa por recursos computacionais passam a ter maior peso em relação ao trabalho útil de cada passo, que, nessa carga, já é pequeno o suficiente para não absorver esse overhead extra.

7.1.2 Speedup

O gráfico de speedup (Figura 3) confirma essa leitura: a curva cresce de forma praticamente linear até 4 threads (speedup em torno de 3.85–3.95x para ambos os schedules, próximo do ideal de 4x), mas o comportamento diverge claramente depois disso. O **dynamic** atinge o pico da carga pequena com 8 threads (speedup de 5.57x) e depois cai para cerca de 4.92x com 16 threads. Já o **static** apresenta uma leve queda de 4 para 8 threads (de $\sim 3.85x$ para $\sim 3.7x$) e volta a subir com 16 threads, superando inclusive o patamar de 4 threads ($\sim 4.0x$) – um comportamento não monotônico que não decorre de nenhuma característica determinística do algoritmo (a corretude foi validada contra a versão sequencial em todos os casos), mas sim de ruído de medição: com uma carga de trabalho pequena e a máquina já sobrecarregada de threads, o tempo de execução fica mais suscetível a interferências do escalonador do SO e de outros processos do sistema, ainda que a média de três execuções tenha sido usada para atenuar esse efeito.

7.1.3 Eficiência

A Figura 3 resume esse cenário de forma bastante clara. A eficiência se mantém próxima de 1.0 até 4 threads para ambos os schedules (0.96 para o `static` e 0.99 para o `dynamic`), mostrando que, nessa faixa, o paralelismo é aproveitado de forma quase ideal. A partir de 8 threads, no entanto, a eficiência despenca: para 0.46 no `static` e 0.70 no `dynamic`, chegando a apenas 0.25 e 0.31, respectivamente, com 16 threads. Isso reforça que, para essa carga de entrada, o uso de threads além do número de núcleos físicos disponíveis (6) apresenta retornos decrescentes: embora o `dynamic` ainda obtenha ganho de desempenho ao passar de 4 para 8 threads, a eficiência cai significativamente, e o aumento para 16 threads não mantém esse ganho. O overhead associado ao escalonamento, à sincronização e à disputa por recursos passa a representar uma parcela maior do custo total, especialmente porque a quantidade de trabalho por passo é relativamente pequena.

7.1.4 Comparação entre os schedules

Em todas as métricas, o `dynamic` (blocos de 512 células) apresenta desempenho superior ao `static` a partir de 4 threads, e essa diferença permanece evidente na região de *oversubscription* (8 e 16 threads). Isso é consistente com a natureza do problema: o custo de `processar_celula` varia célula a célula, sendo maior para células com vizinhos em chamadas (que exigem o cálculo completo do potencial de ignição sobre os 8 vizinhos de Moore) do que para células intactas isoladas ou já queimadas/não combustíveis. Com `schedule(static)`, blocos contíguos e fixos de índices são atribuídos a cada thread no início do laço, de modo que, se a frente de fogo estiver concentrada em uma região da matriz, as threads responsáveis por esses blocos recebem uma carga de trabalho desbalanceada em relação às demais, e o tempo do passo fica limitado pela thread mais lenta. Já o `schedule(dynamic, 512)` redistribui blocos menores sob demanda, permitindo que threads que terminam mais cedo (por processarem regiões com menos células em chamadas) peguem novos blocos, absorvendo parte desse desbalanceamento – ao custo de um overhead adicional de escalonamento e atribuição dos blocos, que tende a compensar quando há de fato heterogeneidade de carga entre regiões da matriz, como é o caso da propagação de incêndio.

7.2 Entrada de Carga Média

A entrada de carga média (1200×1500 células, $P_{max} = 200$) já concentra uma quantidade de trabalho por passo substancialmente maior que a carga pequena, o que se reflete diretamente na forma das curvas.

7.2.1 Tempo de execução

Como mostra a Figura 2, o tempo cai de 8.845 s (sequencial) para cerca de 4.4 s com 2 threads e para aproximadamente 2.25 s com 4 threads, um comportamento muito próximo do ideal e praticamente idêntico entre os dois schedules até esse ponto. A partir de 8 threads os schedules se separam: o `dynamic` chega a seu menor tempo absoluto (1.62 s), enquanto o `static` estabiliza em torno de 1.94 s. Com 16 threads, o `static` melhora discretamente (1.86 s) – alcançando aqui seu menor tempo absoluto entre todas as configurações de threads testadas –, mas o `dynamic` sofre uma leve regressão, subindo para 1.81 s – um aumento de cerca de 12,6% em relação a 8 threads. Esse pequeno recuo é consistente com a queda de speedup do `dynamic` discutida a seguir e, como na carga pequena, é atribuído à disputa

por núcleos em regime de *oversubscription* (8 e 16 threads, acima dos 6 núcleos físicos da máquina) – embora aqui a magnitude seja bem menor que a inversão observada no **static** da carga pequena, já que o volume de trabalho por passo é maior nesta entrada.

7.2.2 Speedup

O speedup (Figura 2) cresce de forma quase linear até 4 threads (~ 3.95 – $4.0x$ para ambos os schedules), confirmando o mesmo padrão observado na carga pequena nessa faixa. O ganho mais expressivo aparece com 8 threads: o **dynamic** atinge seu pico de $5.48x$ (o maior speedup entre as três entradas testadas), enquanto o **static** chega a $4.55x$. Com 16 threads, o **dynamic** tem uma leve queda para $\sim 4.87x$ – refletindo o pequeno aumento de tempo já discutido –, e o **static** continua a subir discretamente, para $\sim 4.8x$, acompanhando a leve melhora de tempo observada para esse schedule.

7.2.3 Eficiência

A Figura 2 mostra eficiência muito próxima de 1.0 até 4 threads (0.99 para o **static** e 1.00 para o **dynamic**), caindo para 0.57 (**static**) e 0.69 (**dynamic**) com 8 threads, e para cerca de 0.30–0.33 com 16 threads. Esses valores são sistematicamente iguais ou levemente superiores aos observados na carga pequena na mesma faixa de threads (especialmente para o **static**, que sai de 0.46 na carga pequena para 0.57 na carga média com 8 threads), um primeiro indício de que o aumento do volume de trabalho por thread melhora o aproveitamento dos recursos paralelos, mesmo com o mesmo hardware.

7.3 Entrada de Carga Grande

A entrada de carga grande (2500×2500 células, $P_{max} = 100$) é a que concentra a maior quantidade de trabalho por passo entre as três, e é onde o comportamento das curvas fica mais previsível e estável.

7.3.1 Tempo de execução

O tempo sequencial de 30.941 s cai para ~ 15.6 s com 2 threads e para ~ 8.03 s (**dynamic**) e ~ 7.98 s (**static**) com 4 threads (Figura 1). Com 8 threads, o **dynamic** atinge 5.96 s e o **static** 6.75 s; com 16 threads, ambos permanecem em patamares próximos (~ 6.06 s e ~ 6.64 s, respectivamente). Diferentemente da carga pequena – onde o **static** chegou a piorar de 4 para 8 threads –, aqui a curva do **static** é monotonicamente decrescente em toda a faixa testada, e a do **dynamic** praticamente estável entre 8 e 16 threads (variação de menos de 1%). Isso é compatível com o fato de que, com uma carga de trabalho grande, a variância relativa associada ao escalonamento do SO e à disputa por recursos em regime de *oversubscription* (8 e 16 threads) representa uma parcela menor do tempo total de execução. Dessa forma, as variações observadas nas cargas menores são atenuadas, embora não desapareçam completamente.

7.3.2 Speedup

O padrão de speedup (Figura 1) repete a forma das outras duas entradas: crescimento quase linear até 4 threads ($\sim 3.9x$), seguido de um salto expressivo até 8 threads (speedup máximo de $5.19x$, com o **dynamic**) e um ganho marginal ou estagnação até 16 threads ($\sim 5.10x$ para

o `dynamic` e $\sim 4.65\times$ para o `static`). O `dynamic` se mantém acima do `static` em toda a faixa a partir de 4 threads, embora a diferença relativa entre os dois seja um pouco menor que na carga pequena.

7.3.3 Eficiência

A eficiência (Figura 1) permanece acima de 0.97 até 4 threads para ambos os schedules, cai para 0.57 (`static`) e 0.65 (`dynamic`) com 8 threads, e para ~ 0.29 e ~ 0.32 com 16 threads. Para o `static`, essa está entre as maiores eficiências observadas entre as três cargas nas faixas de 8 e 16 threads, acompanhando a tendência de que o overhead de paralelização pesa relativamente menos quanto maior a carga de trabalho. Já para o `dynamic`, o padrão não é tão limpo: a eficiência com 8 threads (0.65) é, na verdade, a *menor* entre as três cargas – inferior inclusive à da carga pequena (0.70) –, embora com 16 threads (0.32) volte a ficar em linha com as demais cargas. Uma hipótese para essa exceção é que o overhead associado ao escalonamento e à atribuição dos blocos do `schedule(dynamic, 512)` interaja de forma não uniforme com o tamanho da malha, efeito que, nos resultados obtidos, parece se manifestar com maior intensidade em 8 threads nessa carga. Essa hipótese, contudo, não foi diretamente medida neste experimento.

7.4 Escalabilidade

Colocando as três entradas lado a lado, alguns padrões de escalabilidade ficam evidentes:

- **Escalabilidade quase ideal até 4 threads, independente da carga.** Nas três entradas, a eficiência permanece muito próxima do valor ideal até 4 threads, para os dois schedules – entre 0.96 e 1.02, incluindo pequenos valores acima de 1.0, como 1.017 na carga média com 1 thread, atribuídos a variações de medição em torno do tempo sequencial e insuficientes, neste experimento, para caracterizar superlinearidade real. Isso mostra que, enquanto o número de threads não ultrapassa o número de núcleos físicos disponíveis (6), a estratégia de paralelização – região persistente, divisão do laço de células via `omp for`, reduções para os contadores e `single` apenas para a troca de matrizes – consegue dividir o trabalho quase sem overhead perceptível, independentemente do tamanho da entrada.
- **Queda abrupta de eficiência a partir de 8 threads, também nas três entradas.** A partir de 8 threads (acima dos 6 núcleos físicos da máquina), a eficiência despenca de forma muito parecida nas três cargas: para valores entre 0.46 e 0.70 com 8 threads, e para valores entre 0.25 e 0.33 com 16 threads. Como esse comportamento se repete de forma consistente entre entradas de tamanhos muito diferentes, ele é compatível principalmente com a saturação dos recursos computacionais da máquina: com $T > 6$, múltiplas threads de software disputam os mesmos núcleos físicos, e os custos associados ao escalonamento, à sincronização e à disputa por recursos passam a representar uma parcela maior do tempo de execução em relação ao trabalho útil.
- **Quanto maior a carga, mais estável e previsível a curva – com uma exceção pontual.** A carga pequena é a que apresenta o comportamento mais irregular na faixa de oversubscription (o `static` piorando de 4 para 8 threads, e melhorando de novo em 16), enquanto as cargas média e grande mostram curvas de tempo e speedup mais suaves na maior parte da faixa testada. Ainda assim, a carga média mostra uma

pequena regressão do **dynamic** de 8 para 16 threads (cerca de 12,6%), e a carga grande mostra uma eficiência do **dynamic** com 8 threads inferior à da própria carga pequena – evidenciando que “maior carga = mais estável” é uma tendência geral, não uma regra absoluta para o **dynamic**. Isso é consistente com o fato de a carga pequena ter o menor volume de trabalho por passo entre as três: nela, o tempo de execução é curto o bastante para que a variância do escalonador do sistema operacional e de outros processos concorrentes tenha peso relativo maior sobre a média de três execuções, produzindo as inversões mais pronunciadas. Nas cargas maiores, esse ruído é diluído por um tempo de execução muito mais longo, embora não desapareça por completo, sobretudo para o **dynamic**.

- **A eficiência absoluta do static em 8 e 16 threads melhora com o tamanho da entrada.** Comparando o **static** nas três cargas com 8 threads, a eficiência sobe de 0.46 (pequena) para 0.57 (média e grande); com 16 threads, sobe de 0.25 (pequena) para ~0.29–0.30 (média e grande). Esse ganho é coerente com a redução da importância relativa dos custos não paralelizáveis e dos overheads associados a cada passo: à medida que o volume de trabalho paralelizável aumenta, esses custos representam uma fração menor do tempo total de execução, contribuindo para uma eficiência mais alta nas cargas maiores, mesmo além do ponto de saturação dos núcleos. O mesmo padrão não se sustenta de forma tão limpa para o **dynamic**, cuja eficiência com 8 threads é maior na carga média (0.69) do que na grande (0.65), sugerindo que o overhead de coordenação de blocos desse schedule interage de forma menos previsível com o tamanho da entrada.
- **O dynamic escala melhor que o static nas três cargas, mas a vantagem relativa diminui com o tamanho da entrada.** Na carga pequena, a diferença de eficiência entre **dynamic** e **static** com 8 threads é de 0.70 contra 0.46 (~52% maior); na carga média, de 0.69 contra 0.57 (~21% maior); e na carga grande, de 0.65 contra 0.57 (~14% maior). A vantagem relativa do **dynamic** diminui, portanto, à medida que a carga cresce, embora a tendência não seja monotônica em termos de eficiência absoluta do próprio **dynamic**. Isso sugere que o desbalanceamento de carga entre regiões da matriz (células com muitos vizinhos em chamadas custando mais que células isoladas) tem impacto relativo menor quanto maior a matriz, já que blocos maiores tendem a conter uma mistura mais representativa de células “caras” e “baratas”, aproximando o comportamento do **static** do **dynamic** mesmo sem redistribuição dinâmica de trabalho.
- **O speedup máximo ocorre em 8 threads nas três cargas.** Os valores de pico são 5.57x (pequena), 5.48x (média) e 5.19x (grande), todos abaixo do speedup ideal de 8x para 8 threads, indicando que o escalonamento já deixa de ser linear nessa região. O fato de o pico ocorrer sempre em 8 threads, e não em 4 ou 16, indica que essa configuração ainda proporciona ganho adicional de desempenho, apesar de já estar acima dos 6 núcleos físicos disponíveis. Uma possível explicação é que parte do tempo de execução esteja associada ao acesso à memória, como as leituras dos vizinhos de Moore, permitindo algum aproveitamento adicional de threads. Essa hipótese, contudo, não foi diretamente medida neste experimento, pois não foram coletadas métricas de cache miss, stalls ou uso de banda de memória. Em 16 threads, por outro lado, não há ganho adicional relevante em nenhuma das três entradas.

Em conjunto, esses resultados indicam que a paralelização implementada escala bem

dentro dos limites de hardware da máquina de testes (até 4–6 threads), e que o comportamento observado acima desse limite é compatível principalmente com a saturação dos recursos computacionais, com uma contribuição adicional de ruído de medição – mais pronunciada nas cargas menores e, de forma mais pontual, no `schedule(dynamic)` mesmo em cargas maiores. Os resultados obtidos não indicam, dentro das configurações testadas, uma limitação estrutural evidente da estratégia de paralelização adotada, baseada em *double buffering*, reduções e região paralela persistente.